Tech Challenge: GenAI Engineer

Evidence-Grounded Policy Research Assistant

Section 1: Technical Assessment

Problem Statement

Policy researchers work with government statistics, reports, and internal knowledge that may be incomplete, inconsistent, or difficult to search. Build a GenAI solution that answers analytical questions using supplied or publicly available data, produces traceable evidence, and communicates uncertainty. The challenge prioritises applied machine learning, experimentation, retrieval quality, evaluation, and responsible AI over full-stack engineering.

Challenge Overview

Design and prototype an evidence-grounded policy research assistant. It should ingest a small mixed-format corpus, retrieve relevant evidence, generate cited answers or analytical summaries, and use an evaluation framework to demonstrate where the system succeeds or fails.

You may implement the experience as a notebook, command-line application, lightweight API, or minimal interface. A polished frontend, production-scale backend, multi-cloud integration, and elaborate DevOps setup are not required.

Core Requirements

1. Problem Framing and Experimental Design

- Define the target users, intended decisions, scope, assumptions, and measurable success criteria.
- Identify key failure modes and establish at least one non-LLM or simple baseline.
- Explain how you would split development and evaluation data while avoiding leakage.

2. Data Preparation and Exploratory Analysis

- Use at least two source types, such as structured statistics and unstructured reports. Public, synthetic, or suitably anonymised data may be used.
- Document provenance, licences or usage constraints, schema, missing values, cleaning, transformations, and data-quality checks.

- Perform exploratory analysis and present at least two meaningful findings or visualisations.
- Do not upload confidential, personal, or classified information to external services.

3. Retrieval-Augmented Generation

- Implement an end-to-end ingestion and retrieval pipeline covering parsing, chunking, metadata, indexing, retrieval, and answer generation.
- Compare at least two retrieval or ranking approaches, for example keyword versus dense retrieval, alternative chunking strategies, hybrid search, metadata filtering, or re-ranking.
- Return evidence citations that identify the supporting source and passage.
- Handle unanswerable, ambiguous, conflicting, and out-of-scope questions without fabricating an answer.

4. GenAI Modelling and Experimentation

- Use at least one LLM and justify the model, embedding, prompt, and generation settings.
- Run a controlled comparison of at least two meaningful configurations and analyse quality, latency, and cost trade-offs.
- Use structured outputs where appropriate and validate outputs programmatically.
- Record experiment configurations sufficiently for another person to reproduce the comparison.

5. Agentic or Tool-Using Workflow

- Implement one purposeful agentic workflow in which the model selects or sequences tools, such as retrieval, calculation, dataset querying, or report generation.
- Use deterministic code for calculations and data transformations rather than relying on the LLM to perform arithmetic.
- Show state transitions, tool inputs and outputs, stopping conditions, retry limits, and recovery from at least one tool failure.
- Justify why an agent is appropriate; a bounded workflow is preferred over unnecessary multi-agent complexity.

6. Evaluation and Error Analysis

- Create an evaluation set of at least 30 representative cases, including straightforward, multi-step, unanswerable, conflicting-evidence, and adversarial cases.

- Evaluate retrieval using suitable metrics such as Recall@k, MRR, or nDCG.
- Evaluate generation for groundedness or faithfulness, answer relevance, citation correctness, and task-specific accuracy.
- Include human review of a documented sample and explain the rubric and reviewer process.
- Report latency and estimated cost, stratify results by case type, and conduct qualitative error analysis.
- Identify the highest-impact failure modes and propose evidence-based improvements.

7. Responsible AI, Security, and Privacy

- Test prompt injection, malicious retrieved content, unsupported claims, sensitive-data requests, and unsafe tool invocation.
- Describe controls for permission-aware retrieval, least privilege, secrets, logging, data minimisation, and human oversight.
- Ensure model-generated insights are distinguishable from source facts and statistical findings.

8. GenAI Concepts and Technology

Demonstrate practical understanding of the following concepts and explain relevant trade-offs:

- **LLM fundamentals:** tokenisation, context windows, sampling controls, structured generation, model selection, and common failure modes.
- **Embeddings and retrieval:** similarity measures, chunking, metadata filters, hybrid retrieval, re-ranking, and retrieval-versus-generation errors.
- **Prompt and context engineering:** instruction hierarchy, few-shot examples, context assembly, grounding, prompt versioning, and caching.
- **Evaluation:** offline versus online evaluation, deterministic versus model-based judges, judge bias, test-set leakage, and human review.
- **Agentic systems:** workflow versus agent design, planning, tool calling, state, memory, idempotency, retries, stopping conditions, and human approval.
- **Model Context Protocol (MCP):** the roles of hosts, clients, and servers; tools, resources, and prompts; capability discovery; transport and lifecycle; and when MCP is preferable to direct API integration.
- **Production considerations:** observability, latency, token and cost management, model routing, rate limits, fallbacks, versioning, and reproducibility.

9. MCP and Tool Integration

- Implement or demonstrate a small MCP server exposing at least one useful tool and one resource backed by the challenge data.
- Connect the MCP server to a compatible client or provide an automated protocol-level demonstration if a graphical client is unavailable.
- Define clear input and output schemas, descriptions, validation, errors, and timeouts.
- Show how untrusted tool or resource content is separated from trusted instructions.
- Apply least privilege, explicit user confirmation for consequential actions, secrets protection, and logging with appropriate redaction.
- Discuss trust boundaries, authentication and authorisation, server allow-listing, version compatibility, and the risk of malicious or compromised MCP servers.
- Compare the MCP approach with a direct function or API integration and justify the chosen design.

10. Reproducibility and Minimal Engineering

- Provide a clear setup and execution path, pinned dependencies, configuration templates, and fixed seeds where applicable.
- Include focused tests for data processing, retrieval, output validation, and critical guardrails.
- Containerisation or a lightweight API is welcome but optional unless required for the candidate's chosen design.

Minimum Demonstration Scenarios

1. Answer a factual question with correct source passages and citations.
2. Synthesise evidence across structured and unstructured sources.
3. Perform a calculation using a deterministic tool and cite the underlying data.
4. Abstain or seek clarification when evidence is insufficient or the question is ambiguous.
5. Handle conflicting sources and explain the conflict.
6. Resist a prompt-injection attempt contained in user input or retrieved content.
7. Recover gracefully from a failed retrieval or tool call.
8. Discover and invoke an MCP capability, validate its response, and incorporate the result into a grounded answer.
9. Reject invalid MCP tool arguments and handle a timeout, unavailable server, or malformed response without unsafe fallback behaviour.

10. Identify and ignore an instruction embedded in untrusted MCP resource or tool output.

Submission Requirements

1. Repository or Submission Package

- Source code, notebooks, prompts, configuration templates, and focused tests.
- A README covering the problem, setup, execution, sample queries, limitations, and responsible-use notes.
- An architecture diagram showing data flow, retrieval, model calls, tools, evaluation, and guardrails.
- A data sheet describing sources, provenance, transformations, exclusions, and known quality issues.
- An evaluation report containing the dataset design, metrics, baselines, experiment table, results, error analysis, and improvement plan.
- A short model or system card covering intended use, out-of-scope use, limitations, risks, and mitigations.

2. Live Demonstration

Prepare a 20-minute demonstration followed by questions:

- **End-to-end walkthrough:** show representative questions, evidence retrieval, cited generation, and one analytical or tool-using workflow.
- **Evaluation deep dive:** explain the evaluation set, baseline, experiments, metrics, results, and important errors.
- **Failure demonstration:** show an unanswerable or adversarial case and the system response.
- **Design discussion:** justify modelling choices, trade-offs, safety controls, cost, and next steps.

Technology Choices

You may use any suitable Python data-science and GenAI stack. Cloud-managed or local models are both acceptable. Equivalent tools and frameworks are welcome; assessment is based on reasoning, evidence, and results rather than a prescribed vendor or framework.

MCP may be implemented using an official or community SDK, or directly against the protocol where justified. Candidates should be prepared to explain interoperability, security boundaries, and operational trade-offs rather than relying only on framework abstractions.

Assessment Rubric

Area	Weight	Strong Evidence
Problem Framing and Data Science	10%	Clear hypotheses, appropriate baseline, sound data handling, leakage awareness, and insightful exploratory analysis.
RAG and GenAI Design	20%	Well-justified retrieval, prompting, model choices, citations, abstention, and thoughtful quality-cost-latency trade-offs.
Evaluation and Error Analysis	25%	Representative test set, suitable metrics, reproducible experiments, honest interpretation, and actionable failure analysis.
Agentic Workflow and Tool Use	10%	Purposeful orchestration, deterministic calculations, bounded execution, observable state, and robust failure recovery.
GenAI Concepts and MCP	15%	Accurate conceptual understanding, a well-designed MCP capability, clear schemas and failure handling, secure trust boundaries, and a justified comparison with direct integration.
Responsible AI, Security, and Privacy	10%	Concrete threat testing and effective controls for injection, permissions, sensitive data, tools, auditability, and oversight.
Reproducibility and Communication	10%	Runnable work, clear documentation, focused tests, traceable decisions, and concise communication of limitations.

What We Are Not Optimising For

- A sophisticated frontend or pixel-perfect user experience.
- A large microservice architecture, multi-cloud deployment, or extensive infrastructure.
- The number of agents, frameworks, or model providers used.
- Novel model training without evidence that it improves the task.

Bonus Considerations

- Statistically sound confidence intervals, significance testing, or uncertainty calibration.
- Automated experiment tracking and comparison dashboards.
- Permission-aware retrieval or multilingual evaluation.
- Synthetic-data generation with quality and contamination controls.
- Evaluation of model or embedding drift and a practical monitoring plan.

Section 2: Innovation Assessment

In approximately 1–2 pages, explain how you would evolve the prototype into a trusted GenAI capability for policy researchers in a secure government environment.

Address the following:

- How you would improve retrieval and answer quality using evidence from evaluation.
- How you would detect and respond to changing data, model, or user behaviour.
- How you would support confidence, explainability, provenance, feedback, and human review.
- How you would manage privacy, access controls, auditability, prompt injection, and third-party model risk.
- How you would control latency and cost while preserving quality.
- Which two improvements you would prioritise first and how you would measure their impact.

Candidate Guidance

- Use public, synthetic, or anonymised data only. Do not submit credentials or sensitive information.
- You may mock inaccessible services, but clearly label the mock and explain the production replacement.
- Document assumptions and trade-offs. Negative results and failed experiments are valuable when analysed clearly.
- Use AI-assisted development tools if desired, but be prepared to explain, validate, and take ownership of all submitted work.
- Prioritise a well-evaluated, trustworthy core workflow over breadth.

Evaluation Philosophy

- **Evidence over claims:** support design decisions with experiments and results.
- **Scientific rigour over feature count:** prefer meaningful baselines and analysis to unnecessary complexity.
- **Trustworthiness over fluent output:** citations, abstention, uncertainty, and safeguards matter.
- **Clear thinking over prescribed tools:** explain why your approach suits the problem.
- **Practical impact over novelty:** focus on improvements that help users make better decisions.

Show us how you frame the problem, test hypotheses, measure quality, learn from failures, and build a GenAI system that users can trust.